

Class Diagram

2.1 เขียนโปรแกรม

2.1.1 Vehicle Class

Vehicle Class เป็นคลาสที่ใช้สำหรับเก็บข้อมูลของรถแต่ละคันที่เข้ามาจอดในลานจอดรถ โดยประกอบด้วยข้อมูลดังนี้

- License Plate** — เลขทะเบียนรถ
- Owner Name** — ชื่อเจ้าของรถ
- Entry Time** — เวลาเข้าลานจอดรถ
- Vehicle Type** — ประเภทรถ

ข้อมูลรถแต่ละคันจะถูกสร้างเป็น Object ของ Vehicle และนำไปจัดเก็บใน parkingStack

ตัวอย่างข้อมูล

License Plate : B222
Owner Name : BBB
Entry Time : 11:00
Vehicle Type : Car

2.1.2 ParkingRecord Class

ParkingRecord Class เป็นคลาสที่ใช้สำหรับเก็บข้อมูลผลการทำงานและสถิติของการค้นหาและนำรถออกจากลานจอดรถ

ข้อมูลที่บันทึก ได้แก่

- จำนวน Push
- จำนวน Pop
- จำนวน Comparison
- จำนวนรถที่ถูกย้าย
- สถานะการพบรถเป้าหมาย

ข้อมูลเหล่านี้ใช้สำหรับตรวจสอบการทำงานของ Algorithm และวิเคราะห์ประสิทธิภาพของโปรแกรม

2.1.3 Stack Manager

StackManager เป็นคลาสที่ทำหน้าที่จัดการรถภายในลานจอดรถ โดยใช้ Stack จำนวน 2 ตัว ได้แก่

1. parkingStack

ใช้เป็น Stack หลักสำหรับจัดเก็บรถที่อยู่ภายในลานจอดรถ โดยรถที่เข้ามาที่หลังจะอยู่ด้านบนและถูกนำออกก่อนตามหลักการ **LIFO (Last In First Out)**

2. temporaryStack

ใช้เป็น Stack ชั่วคราวสำหรับเก็บรถที่ขวางรถเป้าหมายระหว่างการค้นหา เมื่อนำรถเป้าหมายออกแล้ว รถใน **temporaryStack** จะถูกนำกลับเข้าสู่ **parkingStack** เพื่อรักษาลำดับของรถที่เหลือ

2.2 Algorithm A

2.2.1 Implement Iteration

Algorithm A ใช้วิธีการทำงานแบบ **Iteration** โดยใช้ **while loop** ในการค้นหารถเป้าหมายจาก **parkingStack**

ขั้นตอนการทำงานมีดังนี้

- 1.ตรวจสอบว่า **parkingStack** ไม่ว่าง
- 2.ใช้ **pop()** นำรถด้านบนออกมา
- 3.เปรียบเทียบ License Plate กับรถเป้าหมาย
- 4.หากไม่ใช่รถเป้าหมาย ให้นำรถไปเก็บใน **temporaryStack**
- 5.หากพบรถเป้าหมาย ให้นำรถออกจากระบบ
- 6.นำรถทั้งหมดจาก **temporaryStack** กลับเข้าสู่ **parkingStack**
- 7.แสดงผลการทำงาน

ตัวอย่าง:

Initial Stack

TOP

D444

C333

B222

A111

BOTTOM

กำหนด Target = B222

Pop D444 → temporaryStack

Pop C333 → temporaryStack

Pop B222 → พบเป้าหมาย

จากนั้นนำรถกลับ:

C333 → parkingStack

D444 → parkingStack

ผลลัพธ์:

TOP

D444

C333

A111

BOTTOM

2.2.2 parkingStack

parkingStack เป็น Stack หลักที่ใช้จัดเก็บรถภายในลานจอดรถ

การเพิ่มรถเข้าสู่ Stack ใช้คำสั่ง:

```
parkingStack.push(vehicle);
```

การนำรถออกจาก Stack ใช้คำสั่ง:

```
parkingStack.pop();
```

ตัวอย่างสถานะ Stack:

TOP

D444

C333

B222

A111

BOTTOM

เมื่อค้นหา **B222** รถที่อยู่ด้านบนคือ **D444** และ **C333** จึงต้องถูกนำออกชั่วคราวก่อน

2.2.3 temporaryStack

temporaryStack เป็น Stack ที่ใช้เก็บรถชั่วคราวระหว่างการค้นหารถเป้าหมาย

ตัวอย่าง:

parkingStack temporaryStack

C333 D444

B222

A111

หลังจากพบ B222 ระบบจะนำรถจาก temporaryStack กลับเข้าสู่ parkingStack

TOP

D444

C333

A111

BOTTOM

วิธีนี้ช่วยให้รถที่เหลือยังคงอยู่ในลำดับเดิม

2.3 ฟังก์ชันระบบ

2.3.1 Menu

โปรแกรมมีเมนูหลักสำหรับควบคุมการทำงานของระบบ ได้แก่

=====

PARKING STACK SYSTEM

=====

1. Add Vehicle
2. Remove Vehicle
3. Display Stack
4. Exit

=====

Choose:

รายละเอียดเมนู

- 1. Add Vehicle** — เพิ่มรถเข้าสู่ลานจอด
- 2. Remove Vehicle** — ค้นหาและนำรถเป้าหมายออก
- 3. Display Stack** — แสดงสถานะของ Stack
- 4. Exit** — ออกจากโปรแกรม

2.3.2 Input Validation

ระบบมีการตรวจสอบข้อมูลที่ผู้ใช้ป้อน โดยไม่อนุญาตให้ข้อมูลที่จำเป็นเป็นค่าว่าง

หากผู้ใช้ไม่กรอกข้อมูล ระบบจะแจ้งเตือนและให้กรอกใหม่ เช่น

Input cannot be empty. Please try again.

นอกจากนี้ระบบยังตรวจสอบทะเบียนรถซ้ำก่อนเพิ่มรถเข้าสู่ **parkingStack**

2.3.3 แสดงสถานะ Stack

โปรแกรมสามารถแสดงรถที่อยู่ใน Stack ตั้งแต่ **TOP** ไปจนถึง **BOTTOM**

ตัวอย่าง:

===== Parking Stack =====

TOP

D444 | DDD | 13:00 | Car

C333 | CCC | 12:00 | Car

B222 | BBB | 11:00 | Car

A111 | AAA | 10:00 | Car

BOTTOM

2.3.4 Count Push

โปรแกรมมีการนับจำนวนครั้งที่มีการใช้คำสั่ง **push()** เพื่อเพิ่มข้อมูลเข้าสู่ Stack

ตัวอย่างกรณี **Target = B222**

D444 → temporaryStack

C333 → temporaryStack

C333 → parkingStack

D444 → parkingStack

ดังนั้น

Push = 4

2.3.5 Count Pop

โปรแกรมมีการนับจำนวนครั้งที่มีการใช้คำสั่ง **pop()** เพื่อนำรถออกจาก Stack

กรณี **Target = B222**

Pop D444

Pop C333

Pop B222

ดังนั้น

Pop = 3

2.3.6 Count Comparison

โปรแกรมมีการนับจำนวนครั้งที่เปรียบเทียบ License Plate ของรถกับรถเป้าหมาย

กรณี **Target = B222**

D444 $\neq$ B222 $\rightarrow$ Comparison 1

C333 $\neq$ B222 $\rightarrow$ Comparison 2

B222 = B222 $\rightarrow$ Comparison 3

ดังนั้น

Comparison = 3

2.4 Class Diagram และ UML

โครงสร้างหลักของโปรแกรมประกอบด้วย 4 คลาส ได้แก่

-Main

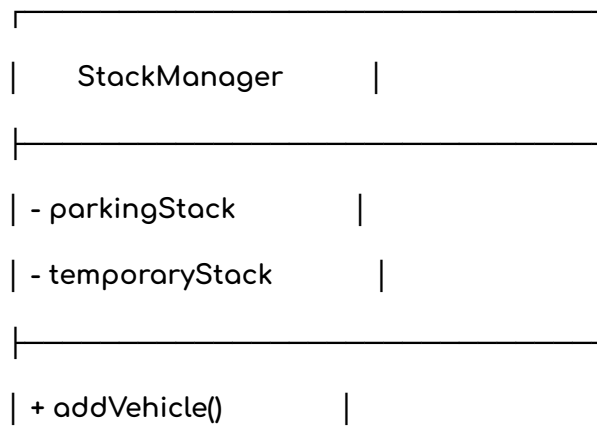
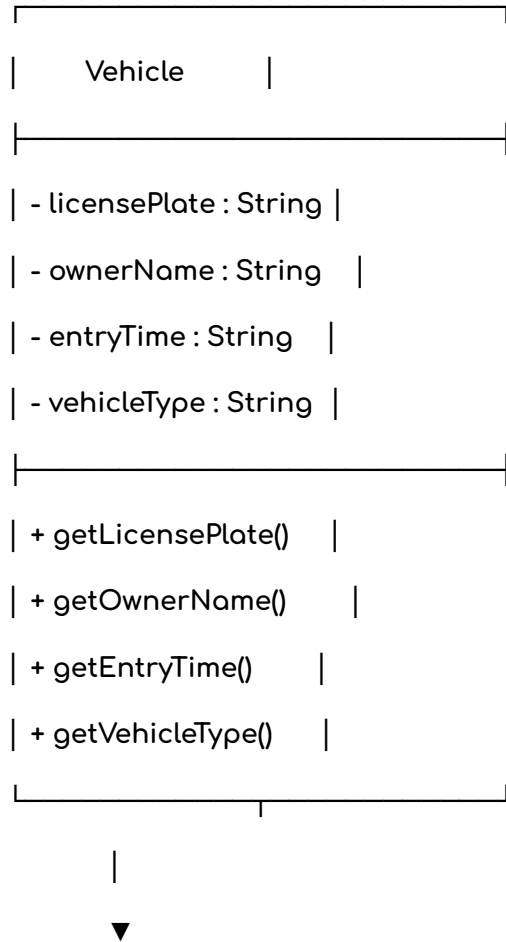
-Vehicle

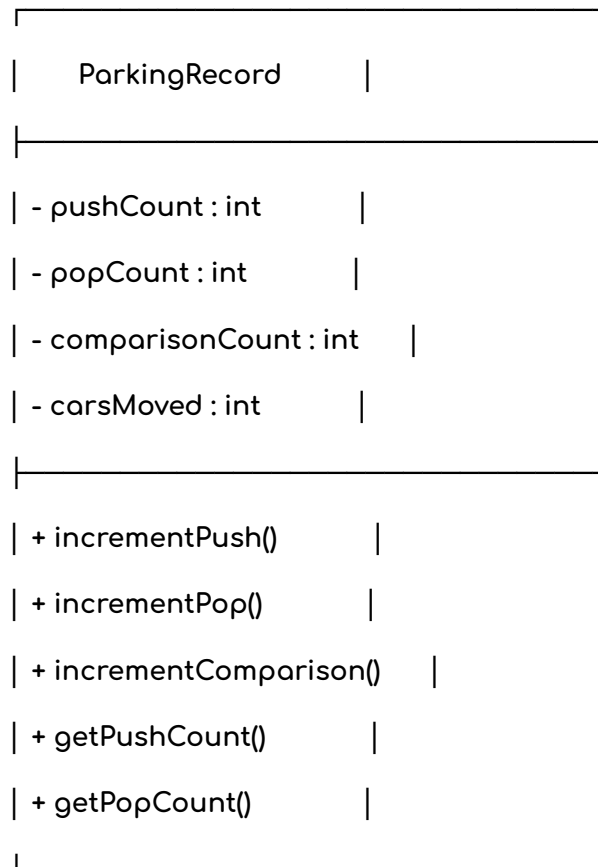
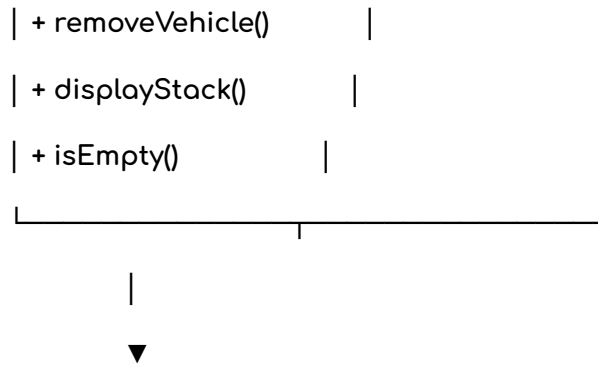
-ParkingRecord

-StackManager

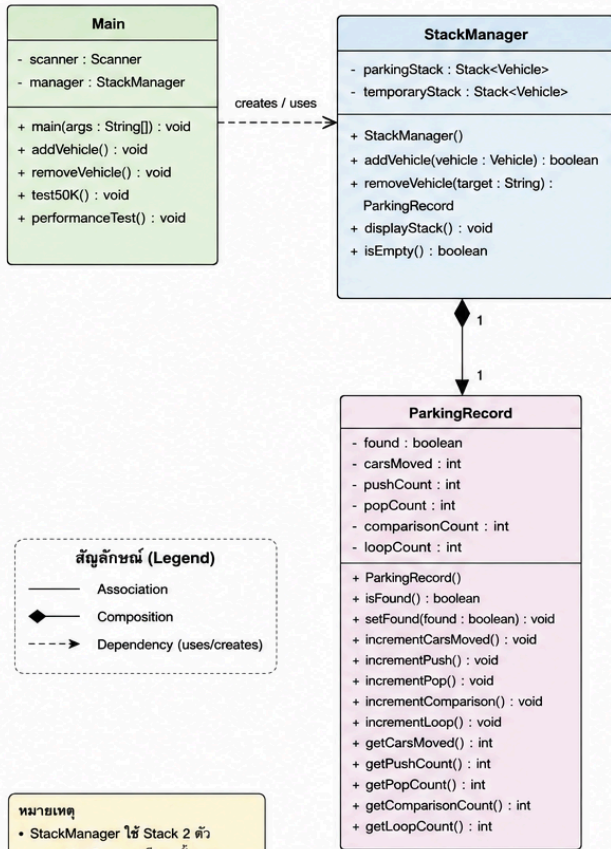
โดย **Main** ทำหน้าที่ควบคุมการทำงานของโปรแกรม **Vehicle** ใช้เก็บข้อมูลรถ **StackManager** จัดการ Stack และ **ParkingRecord** ใช้เก็บสถิติการทำงานของ Algorithm

Class Diagram





Class Diagram - Parking Stack System



สัญลักษณ์ (Legend)

- Association
- ◆ Composition
- .-> Dependency (uses/creates)

หมายเหตุ

- StackManager ใช้ Stack 2 ตัว
 - parkingStack : เก็บรถทั้งหมด
 - temporaryStack : ใช้พักชั่วคราว
- Algorithm ที่ใช้ในการลบรถ : Algorithm A (Iteration)
- สมมุติฐาน : ป้ายทะเบียนไม่ซ้ำกัน

Flowchart - Parking Stack System (Algorithm A : Iteration)

